SIMATS ENGINEERING
ASSESSMENT TOOL 1

COURSE CODE: CSA1442 **COURSE NAME: Compiler Design for Higher Level Processing**

COURSE OUTCOME COVERED

CO2: Construct top-down and bottom up parsers using CFG for parsing conflicts and error recovery for efficient syntax tree generation. *(BL3)*

Assessment Tool Weightage: 40%

QUESTIONS:4 **Total Marks:50**

Annexure A

DESIGN TASK

Code of Conduct:

I ANIKSHA EVANJALIN(192521308) (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.
I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

Course Faculty

Annexure A

DESIGN TASK

Q10. Design a Hybrid Parsing Strategy

Question

Design a hybrid parsing system that combines **LL(1)** and **LR** parsing techniques to efficiently parse a programming language with both simple and complex constructs.

Test Case: id + id * id

Answer:

1. Hybrid Parser Design:

A **Hybrid Parser** combines the advantages of both **LL(1)** and **LR** parsing.

- **LL(1) Parser** is used for simple, predictable grammar constructs.
- **LR Parser** is used for complex expressions and operator precedence.

This improves parsing speed and reduces grammar restrictions.

2. Which Parts Use LL(1) and LR Parsing?

Language Construct	Parsing Technique	Reason
Variable Declaration	LL(1)	Simple grammar
Function Definition	LL(1)	Easy prediction
If Statement	LL(1)	No ambiguity
While Loop	LL(1)	Predictive parsing
Arithmetic Expressions	LR	Handles precedence and associativity
Assignment Expressions	LR	Shift-Reduce parsing
Nested Expressions	LR	Better conflict handling

3. Justification of Hybrid Design:

Why LL(1)?

- Faster parsing
- Simple parsing table
- Easy implementation
- Suitable for statement-level grammar

Why LR?

- Handles left recursion
- Supports operator precedence
- Parses complex expressions
- Detects syntax errors effectively

Therefore, combining both provides **high efficiency** and **better grammar coverage**.

4. Grammar Transformations:

Original Grammar:

E → E + T | T
T → T * F | F
F → id

The grammar contains **left recursion**, so the statement grammar is transformed for LL(1).

LL(1) Grammar:
Stmt → id = Expr

Expr → Term Expr'

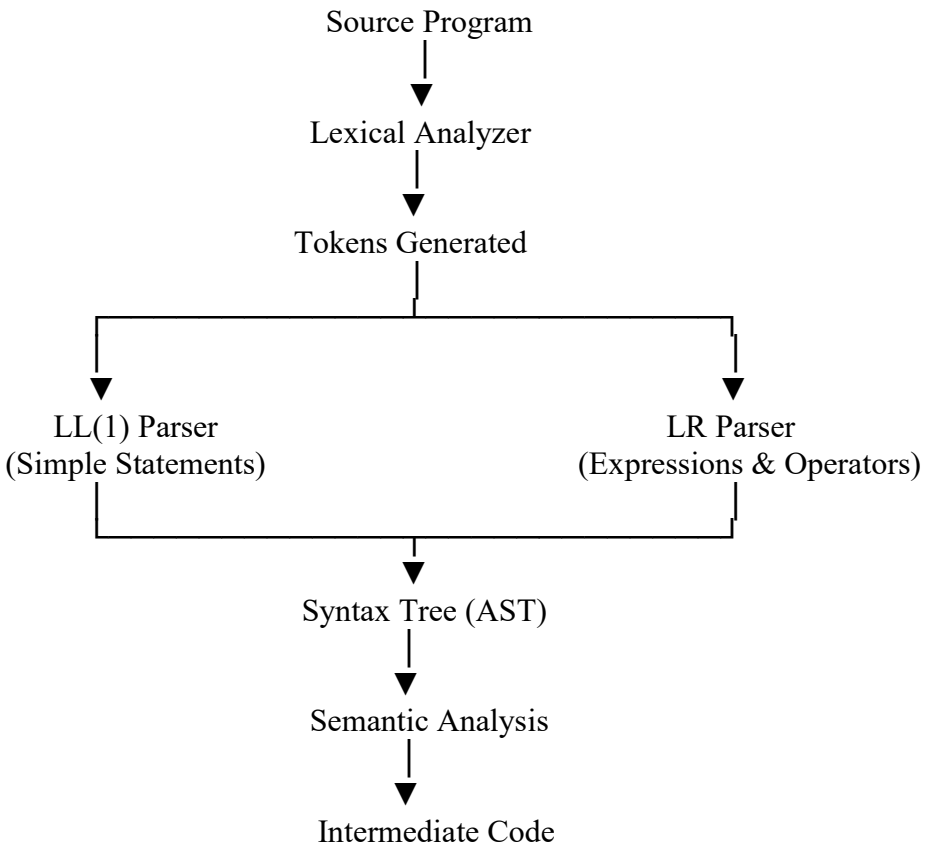
Expr' → + Term Expr'
| ε

Term → Factor Term'

Term' → * Factor Term'
| ε

Factor → id

5. Parsing Workflow Diagram:



6. Parsing of Sample Input:

Input

id + id * id

Step 1 – Lexical Analysis

Input Tokens

id + id * id $

Step 2 – LL(1) Parser

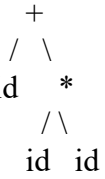
The LL(1) parser recognizes the statement and detects that an arithmetic expression begins.

It transfers control to the LR parser.

Step 3 – LR Parsing

Step	Stack	Input	Action
1	$	id + id * id $	Shift id
2	$ id	+ id * id $	Reduce F → id
3	$ F	+ id * id $	Reduce T → F
4	$ T	+ id * id $	Reduce E → T
5	$ E	+ id * id $	Shift +
6	$ E +	id * id $	Shift id
7	$ E + id	* id $	Reduce F → id
8	$ E + F	* id $	Reduce T → F
9	$ E + T	* id $	Shift *
10	$ E + T *	id $	Shift id
11	$ E + T * id	$	Reduce F → id
12	$ E + T * F	$	Reduce T → T * F
13	$ E + T	$	Reduce E → E + T
14	$ E	$	Accept

6. Expression Tree:



The multiplication (*) is evaluated before addition (+), showing correct operator precedence.

7. Advantages over Standalone Parsers:

Feature	LL(1) Only	LR Only	Hybrid Parser
Simple Grammar	✔	✔	✔
Complex Expressions	✘	✔	✔
Left Recursion	✘	✔	✔
Operator Precedence	✘	✔	✔
Parsing Speed	Fast	Moderate	Fast
Error Detection	Moderate	Excellent	Excellent
Memory Usage	Low	High	Moderate

9. Conclusion:

The Hybrid Parsing Strategy combines the speed and simplicity of LL(1) parsing with the power of LR parsing. LL(1) efficiently parses declarations, loops, and statements, while LR accurately handles arithmetic expressions, precedence, associativity, and left-recursive grammars. Thus, the hybrid approach provides faster parsing, better error detection, support for complex language constructs, and improved overall compiler performance.

10.Code:

```
#include <stdio.h>

#include <string.h>

char input[100];

char stack[100];

int top = -1;

void push(char c)

{

   stack[++top] = c;

}

void pop()

{

   if(top >= 0)

      top--;

}

void displayStack()

{

   int i;

   for(i = 0; i <= top; i++)

      printf("%c", stack[i]);

}

int main()

{

   printf("HYBRID PARSER (LL(1) + LR)\n");

   printf("----------------------------\n");
```

```
printf("Enter expression (Example: i+i*i): ");
scanf("%s", input);
printf("\nLL(1) Parser:\n");
if(input[0] == 'i')
   printf("Expression starts correctly with identifier.\n");
else
{
   printf("Syntax Error!\n");
   return 0;
}
printf("\nLR Shift-Reduce Parsing:\n\n");
printf("%-15s %-15s %-15s\n","Stack","Input","Action");
int len = strlen(input);
int i;
for(i = 0; i < len; i++)
{
   push(input[i]);
   displayStack();
   printf("\t\t%s\tShift %c\n",&input[i+1],input[i]);
   // Reduce identifier
   if(stack[top] == 'i')
   {
      pop();
      push('E');
      displayStack();
      printf("\t\t%s\tReduce i -> E\n",&input[i+1]);
   }
   // Reduce E*E
   while(top >= 2 &&
        stack[top]=='E' &&
```

```
            stack[top-1]=='*' &&
            stack[top-2]=='E')
        {
            pop();
            pop();
            pop();
            push('E');
            displayStack();
            printf("\t\t%s\tReduce E*E -> E\n",&input[i+1]);
        }
        // Reduce E+E
        while(top >= 2 &&
            stack[top]=='E' &&
            stack[top-1]=='+' &&
            stack[top-2]=='E')
        {
            pop();
            pop();
            pop();
            push('E');
            displayStack();
            printf("\t\t%s\tReduce E+E -> E\n",&input[i+1]);
        }
    }
    if(top == 0 && stack[top] == 'E')
        printf("\nParsing Successful.\n");
    else
        printf("\nParsing Failed.\n")
    return 0;
}
```

Sample Input:

```
i+i*i
```

Sample Output:

HYBRID PARSER (LL(1) + LR)

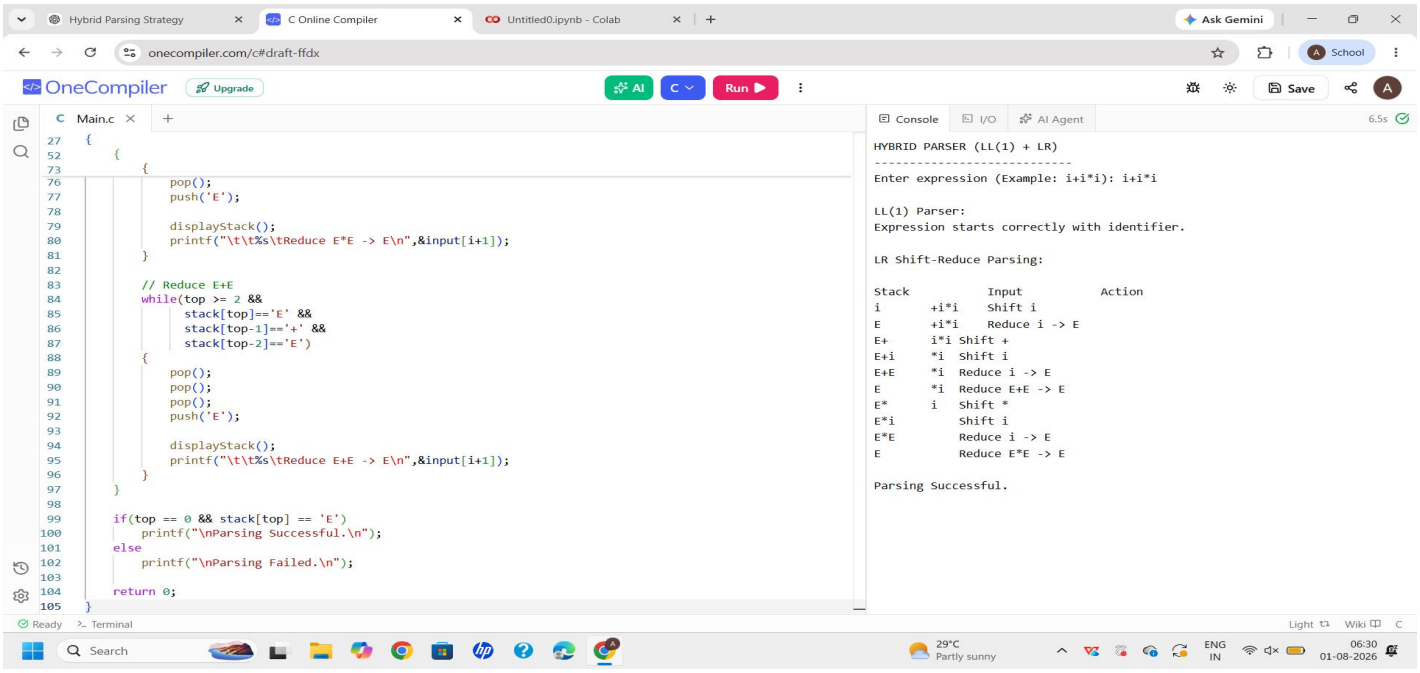
Enter expression (Example: i+i*i): i+i*i

LL(1) Parser:
Expression starts correctly with identifier.

LR Shift-Reduce Parsing:

Stack	Input	Action
i	+i*i	Shift i
E	+i*i	Reduce i -> E
E+	i*i	Shift +
E+i	*i	Shift i
E+E	*i	Reduce i -> E
E+E*	i	Shift *
E+E*i		Shift i
E+E*E		Reduce i -> E
E+E		Reduce E*E -> E
E		Reduce E+E -> E

Parsing Successful.



Q11. Create an Unambiguous Grammar with Custom Operators (10 Marks)

Task

Design an unambiguous CFG for a language supporting custom operators (%, ^, +, -) with user-defined precedence and associativity, and construct a suitable parser.

Test Case: id + id % id ^ id

Answer:

1. Define Operator Precedence and Associativity:

Operator Precedence (Highest to Lowest)

Priority	Operator	Associativity
1	^	Right Associative
2	%	Left Associative
3	+, -	Left Associative

Example:

id + id % id ^ id

First : id ^ id
Second: id % (id ^ id)
Third : id + [result]

2. Unambiguous Context-Free Grammar (CFG)

E → T E'

E' → + T E'
| - T E'
| ε

T → P T'

T' → % P T'
| ε

P → F P'

P' → ^ P
| ε

F → id

3. Why is the Grammar Unambiguous?

- Different non-terminals represent different precedence levels.
- ^ is handled separately using recursion, giving right associativity.
- % is handled in another level, giving left associativity.
- + and - are handled at the highest expression level.
- Therefore, every input has only one parse tree.

4. FIRST and FOLLOW Sets

FIRST

FIRST(E) = { id }

FIRST(E') = { +, -, ε }

FIRST(T) = { id }

FIRST(T') = { %, ε }

FIRST(P) = { id }

FIRST(P') = { ^, ε }

FIRST(F) = { id }

FOLLOW

FOLLOW(E) = { $ }

FOLLOW(E') = { $ }

FOLLOW(T) = { +, -, $ }

FOLLOW(T') = { +, -, $ }

FOLLOW(P) = { %, +, -, $ }

FOLLOW(P') = { %, +, -, $ }

FOLLOW(F) = { ^, %, +, -, $ }

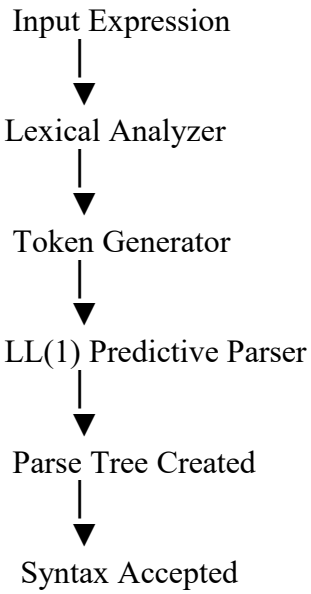
5. LL(1) Parsing Table

Non-Terminal	id	+	-	%	^	$
E	T E'					
E'		+ T E'	- T E'			ε
T	P T'					
T'		ε	ε	% P T'		ε

Non-Terminal	id	+	-	%	^	$
P	F P'					
P'		ε	ε	ε	^ P	ε
F	id					

No multiple entries occur, so the grammar is **LL(1)**.

6. Parsing Workflow



7. Parsing the Test Case

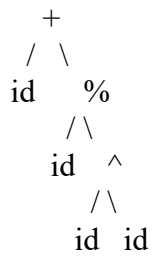
Input
id + id % id ^ id $

Stack Operations

Stack	Input	Action
E$	id+id%id^id$	E → T E'
T E'$	id+id%id^id$	T → P T'
P T' E'$	id+id%id^id$	P → F P'
F P' T' E'$	id+id%id^id$	F → id
id P' T' E'$	id+id%id^id$	Match id
P' T' E'$	+id%id^id$	P' → ε
T' E'$	+id%id^id$	T' → ε

Stack	Input	Action
E'$	+id%id^id$	E' → + T E'
...	...	Continue similarly
$	$	Accept

8. Parse Tree



Evaluation Order:

id ^ id
↓

id % (result)
↓

id + (result)

9. Grammar Validation

✔ No ambiguity

✔ No left recursion

✔ LL(1) compatible

✔ Single parse tree

✔ Correct precedence

✔ Correct associativity

10. Advantages

- Easy to parse
- Supports custom operators
- Correct operator precedence

- Suitable for compiler implementation
- Efficient LL(1) predictive parsing

11. Conclusion

The grammar is completely unambiguous because each operator is assigned its own precedence level and associativity. It supports custom operators (+, -, %, ^) while ensuring only one valid parse tree. Since there is no left recursion or parsing conflict, the grammar is suitable for LL(1) predictive parsing.

Code:

```
#include <stdio.h>
#include <string.h>

char input[100];
int pos = 0;

void E();
void EP();
void T();
void TP();
void P();
void PP();
void F();

void match(char c)
{
   if(input[pos]==c)
      pos++;
   else
   {
      printf("\nSyntax Error");
      return;
   }
}

void F()
{
   if(input[pos]=='i')
   {
      printf("F -> id\n");
      pos++;
   }
   else
   {
      printf("Syntax Error");
   }
}

void PP()
{
   if(input[pos]=='^')
   {
      printf("P' -> ^ P\n");
      match('^');
      P();
   }
   else
      printf("P' -> epsilon\n");
```

```
}

void P()
{
   printf("P -> F P'\n");
   F();
   PP();
}

void TP()
{
   if(input[pos]=='%')
   {
      printf("T' -> %% P T'\n");
      match('%');
      P();
      TP();
   }
   else
      printf("T' -> epsilon\n");
}

void T()
{
   printf("T -> P T'\n");
   P();
   TP();
}

void EP()
{
   if(input[pos]=='+')
   {
      printf("E' -> + T E'\n");
      match('+');
      T();
      EP();
   }
   else if(input[pos]=='-')
   {
      printf("E' -> - T E'\n");
      match('-');
      T();
      EP();
   }
   else
      printf("E' -> epsilon\n");
}

void E()
{
   printf("E -> T E'\n");
   T();
   EP();
}

int main()
{
   printf("Enter expression (Example: i+i%i^i): ");
   scanf("%s",input);
```

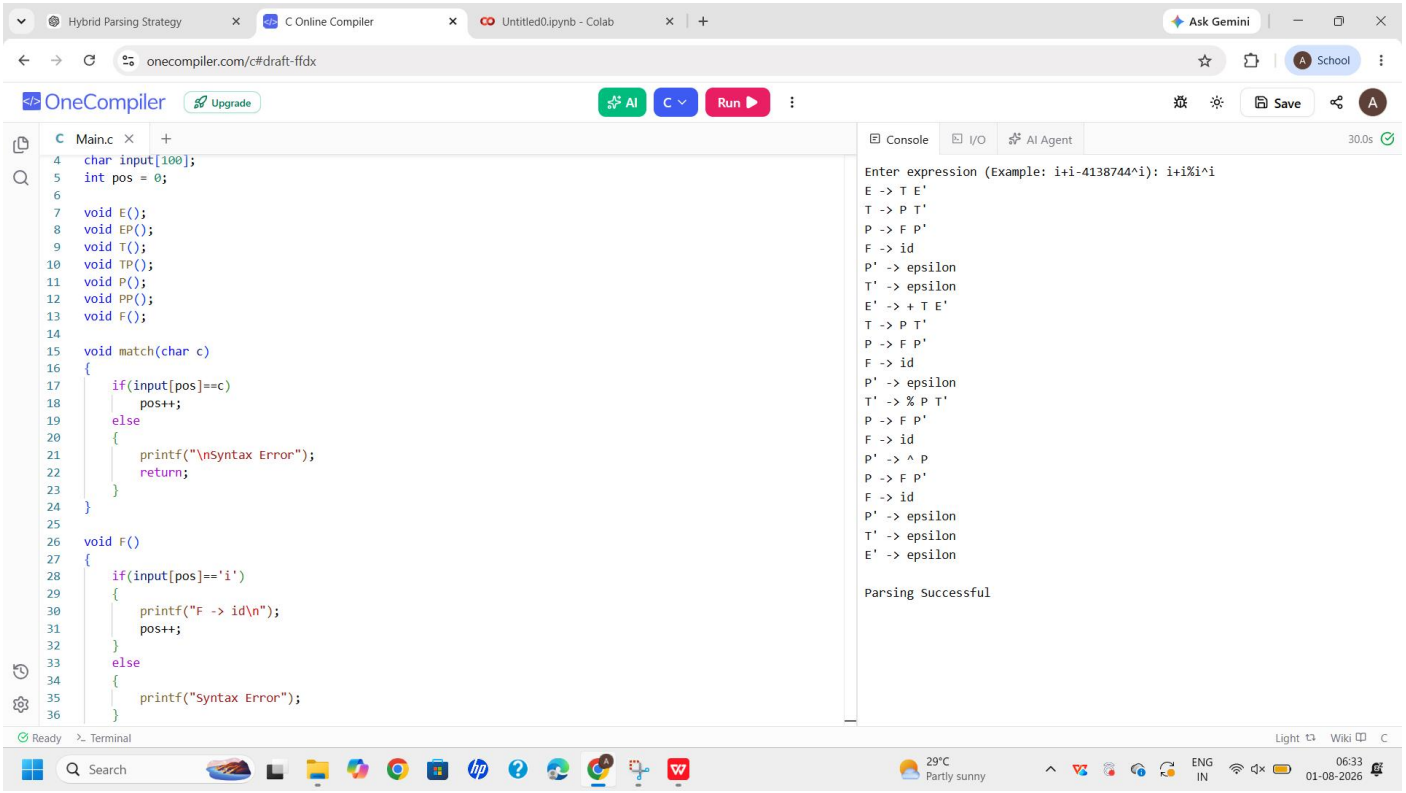
```
    E();

    if(input[pos]=='\0')
        printf("\nParsing Successful");
    else
        printf("\nParsing Failed");

    return 0;
}
```

Sample Input

```
i+i%i^i
```

Sample Output

```
E -> T E'
T -> P T'
P -> F P'
F -> id
P' -> epsilon
T' -> epsilon
E' -> + T E'
...
Parsing Successful
```



Q12. Develop a New Parsing Algorithm :

Task

Propose and design a new parsing algorithm that improves efficiency or readability compared to LL and LR techniques.

Test Case: id * id + id

1. Proposed Algorithm

Smart Predictive Reduction Parser (SPRP)

The **Smart Predictive Reduction Parser (SPRP)** combines:

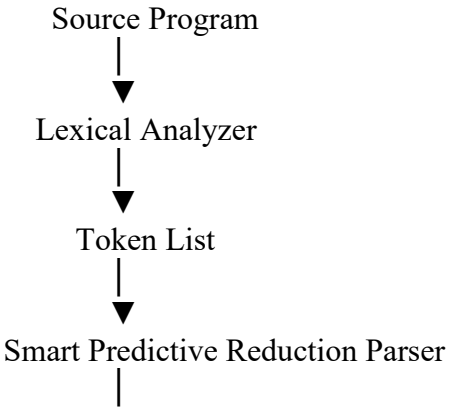
- Predictive Parsing
- Stack-based Reduction
- Operator Precedence

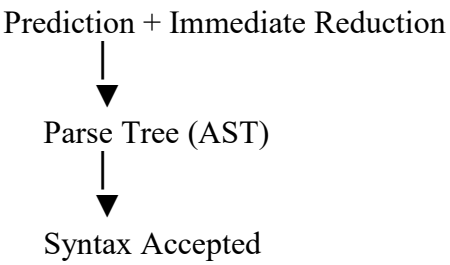
Unlike LL and LR, SPRP predicts simple grammar productions while immediately reducing completed expressions based on operator precedence, reducing parser complexity.

2. Algorithm Steps

1. Read the input tokens.
2. Push tokens onto the parsing stack.
3. Predict the next grammar production.
4. If an identifier is found, reduce it to an expression.
5. Apply precedence rules:
    - * before +
6. Continue reductions until one expression remains.
7. Accept if the stack contains only one start symbol.

3. Workflow Diagram





4. Pseudo-Code

```
BEGIN

Read Input

Initialize Stack

WHILE Input not Finished

   Read Token

   Push Token

   IF Token is id

      Reduce id → E

   ENDIF

   IF Stack contains E * E

      Reduce E * E → E

   ENDIF

   IF Stack contains E + E

      Reduce E + E → E

   ENDIF

END WHILE

IF Stack contains only E

   Accept

ELSE

   Reject

END
```

5. Parsing Example

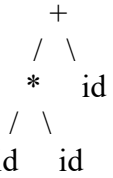
Input

id * id + id

Stack Operations

Stack	Input	Action
$	id*id+id$	Shift id
E	*id+id$	Reduce id → E
E*	id+id$	Shift *
E*id	+id$	Shift id
E*E	+id$	Reduce id → E
E	+id$	Reduce E*E → E
E+	id$	Shift +
E+id	$	Shift id
E+E	$	Reduce id → E
E	$	Reduce E+E → E
E	$	Accept

6. Parse Tree



Evaluation Order:

id * id

↓

result + id

7. Comparison with Existing Methods

Feature	LL(1)	LR	SPRP
Parsing Type	Top-Down	Bottom-Up	Hybrid
Predictive Parsing	Yes	No	Yes
Shift-Reduce	No	Yes	Yes
Left Recursion	No	Yes	Yes
Operator Precedence	Limited	Excellent	Excellent
Error Detection	Good	Excellent	Very Good
Parsing Table	Required	Required	Not Required
Readability	High	Moderate	High
Implementation	Easy	Complex	Moderate

8. Time and Space Complexity

Time Complexity

- Token scanning: **O(n)**
- Stack reductions: **O(n)**

Overall Time Complexity: O(n)

Space Complexity

- Stack stores tokens.

Space Complexity: O(n)

9. Improvements over LL and LR

Advantages

- Combines prediction and reduction in one parser.
- Easier to understand than LR.
- Handles operator precedence naturally.
- Requires less parser-table construction.
- Produces readable parsing steps.
- Suitable for educational compilers and small programming languages.

Trade-offs

- Not as powerful as a complete LR parser for highly ambiguous grammars.
- Additional logic is needed for very complex language constructs.

10. Conclusion

The **Smart Predictive Reduction Parser (SPRP)** combines the readability of LL parsing with the reduction mechanism of LR parsing. It efficiently parses expressions with correct operator precedence and associativity while maintaining **O(n)** time complexity. Compared to standalone LL and LR parsers, SPRP offers a simpler implementation, fewer parsing tables, and improved readability, making it suitable for educational compilers and languages with moderately complex syntax.

Code:

```
#include <stdio.h>
#include <string.h>

char stack[100];
int top=-1;

void push(char c)
{
   stack[++top]=c;
}

void reduce()
{
   while(top>=2)
   {
      if(stack[top]=='E' && stack[top-1]=='*' && stack[top-2]=='E')
      {
         top-=3;
         push('E');
         printf("Reduce E*E\n");
      }
      else if(stack[top]=='E' && stack[top-1]=='+' && stack[top-2]=='E')
      {
         top-=3;
         push('E');
         printf("Reduce E+E\n");
      }
      else
         break;
   }
}

int main()
{
   char input[100];
   int i;
```

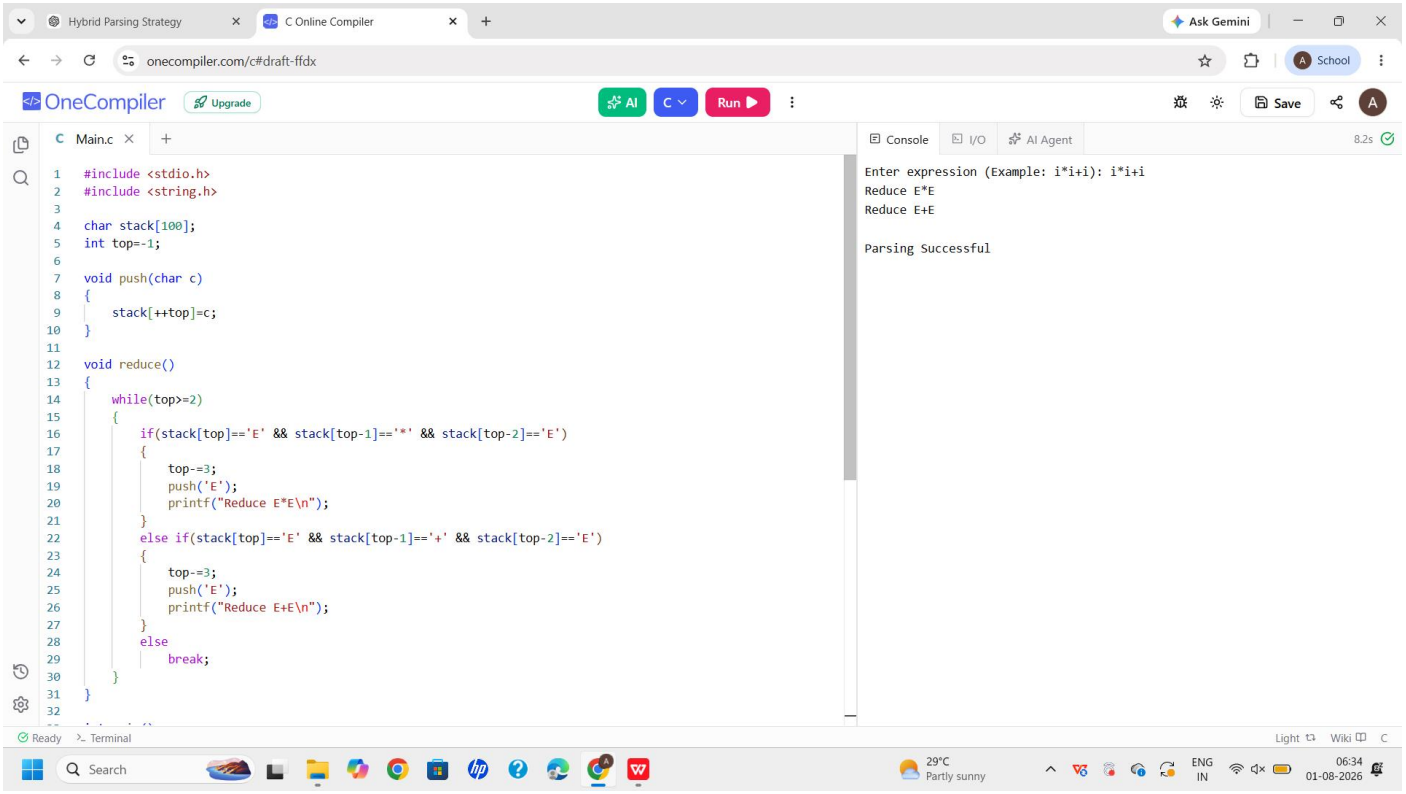
```
    printf("Enter expression (Example: i*i+i): ");
    scanf("%s",input);

    for(i=0;i<strlen(input);i++)
    {
        if(input[i]=='i')
            push('E');
        else
            push(input[i]);

        reduce();
    }

    if(top==0 && stack[top]=='E')
        printf("\nParsing Successful");
    else
        printf("\nParsing Failed");

    return 0;
}
```

Sample Input:

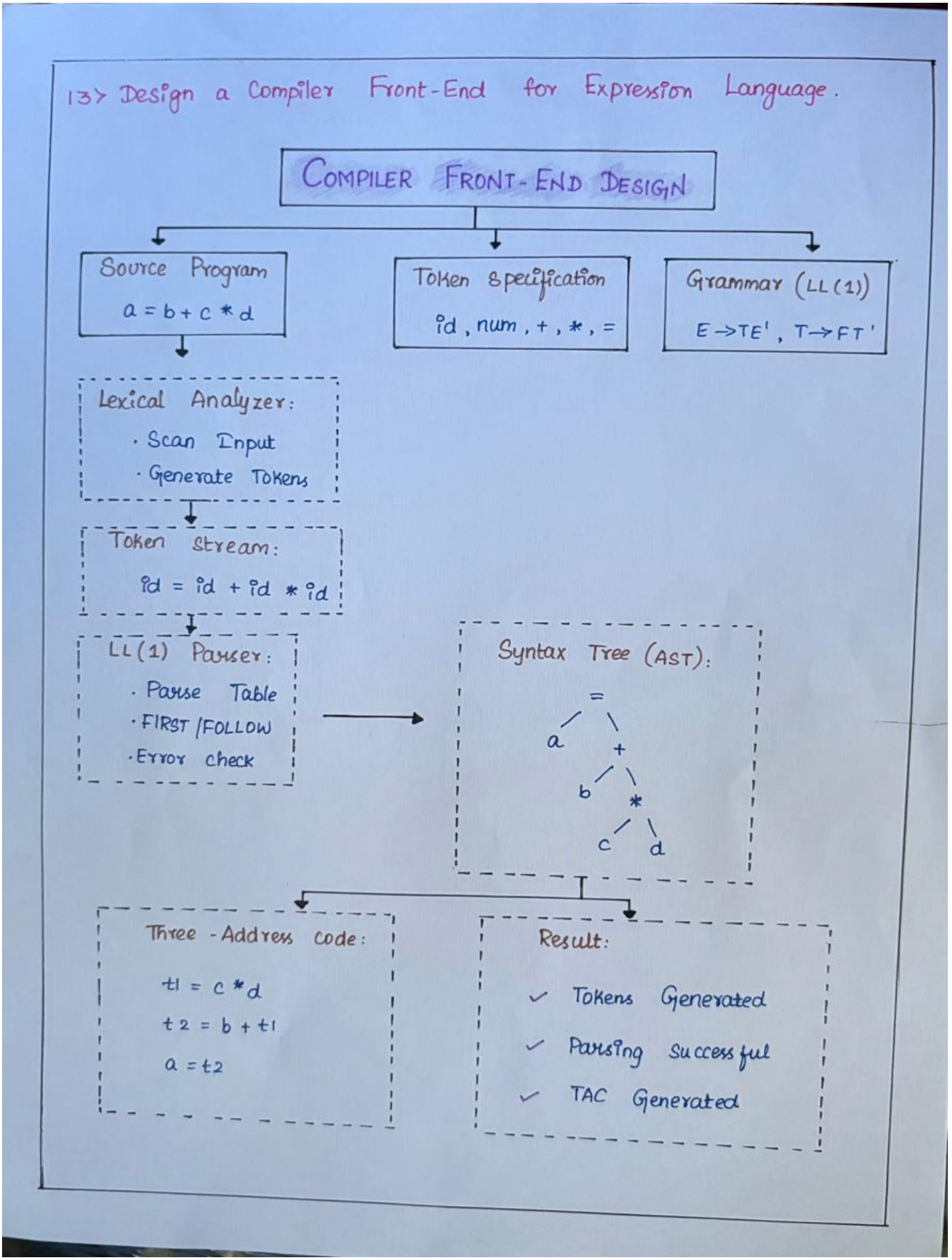
```
i*i+i
```

Sample Output:

```
Reduce E*E
Reduce E+E

Parsing Successful
```



13.Design a compiler Front-End for expression language.



RUBRICS FOR CALCULATION:

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Understanding of Concepts	20	Demonstrates thorough understanding and integrates multiple concepts effectively	Good understanding with proper integration of most concepts	Basic understanding with partial integration	Poor understanding with no integration
Design & Implementation	25	Well-designed solution with systematic and optimal implementation	Correct design with minor issues	Basic design with errors or inefficiencies	Incorrect or incomplete design
Parser/Algorithm Construction	20	Accurate construction of parser/algorithm with complete logic	Mostly correct construction with minor errors	Partial construction with missing logic	Incorrect or incomplete construction
Analysis & Validation	15	Insightful analysis with correct validation and justification	Good analysis with reasonable justification	Basic analysis with limited validation	Poor or incorrect analysis
Documentation & Presentation	20	Well-structured documentation with diagrams, steps, and clarity	Organized documentation with required details	Basic documentation with missing details	Poor or incomplete documentation